

Кортеж





Что такое кортеж?

Кортежи (тип tuple) — это неизменяемый тип данных в Python, который используется для хранения упорядоченной последовательности элементов. У этих коллекций есть три замечательных свойства:

- **Неизменяемость.** После того как кортеж создан, в него нельзя добавлять элементы, а также изменять их или удалять.
- **Упорядоченность.** Элементы кортежа располагаются в определённом порядке, который тоже неизменяем. К любому элементу можно обратиться по его индексу (порядковому номеру).
- **Элементами кортежа могут быть объекты разных типов данных:** числа, строки, списки, другие кортежи и другие.

Кортежи записываются с использованием круглых скобок ():

```
my_tuple = (значение1, значение2...)
```



Кортеж , для чего можно использовать

Применение кортежей связано с их свойствами — неизменяемостью и строгим порядком элементов. Разберём ситуации, когда это полезно.

Требуется защитить элементы от изменений.

Если нужно защитить данные от случайного или намеренного изменения, то можно использовать кортежи. Например, этот тип данных подойдёт для сохранения информации о датах рождения клиентов. Обратите внимание: неизменяемость кортежей не абсолютна. У неё есть исключение — если внутри кортежа находятся изменяемые элементы, например списки, словари или множества, то их значения можно изменить.

```
# Кортеж с изменяемыми элементами:
mutable_tuple = ([1, 2, 3], [4, 5, 6])

print('Исходный кортеж:', mutable_tuple)

# Изменение содержимого списка внутри кортежа
mutable_tuple[0][1] = 10 # Меняем второй элемент в первом списке
mutable_tuple[1].append(7) # Добавляем ко второму списку новое значение

print('Изменённый кортеж:', mutable_tuple)
```

>>>

```
Исходный кортеж: ([1, 2, 3], [4, 5, 6])
Изменённый кортеж: ([1, 10, 3], [4, 5, 6, 7])
```



Кортеж , для чего можно использовать

Необходимо экономить память

Кортежи занимают в памяти меньше места, чем списки. Это легко проверить:

```
a = (10, 20, 30, 40, 50, 60) # Кортеж
b = [10, 20, 30, 40, 50, 60] # Список

>>> 72
print(a.__sizeof__()) # Размер кортежа
88
print(b.__sizeof__()) # Размер списка
```

Кортеж занимает в памяти на 16 байт меньше, чем список. Из-за малой величины сравниваемых объектов разница почти незаметна, но при работе с данными большего объёма экономия может быть значимой.

Предстоит работать со словарями

Кортежи идеально подходят для использования в качестве ключей в словарях, так как ключи в них должны быть объектами неизменяемого типа данных. Проверим это:

```
# Создаём словарь с кортежем в качестве ключа
my_dict = {('a', 1): 'value1', ('b', 2): 'value2'}
# Обращаемся к элементам словаря по кортежу в качестве ключа
print(my_dict[('a', 1)]) # Вывод: value1
print(my_dict[('b', 2)]) # Вывод: value2
# Добавляем новый элемент с кортежем в качестве ключа
my_dict[('c', 3)] = 'value3'
# Выводим содержимое словаря
print(my_dict)
```

>>> value1
value2
{('a', 1): 'value1', ('b', 2): 'value2', ('c', 3): 'value3'}

Операции над кортежами в Python





Основные операции над кортежами

Создание кортежа

Создать кортеж в Python можно по меньшей мере пятью способами. Рассмотрим каждый из них:

- С помощью круглых скобок: `my_tuple = (1, 2, 3, 'a', 'b', 'c')`
- Без круглых скобок: `my_tuple = 1, 2, 3, 'a', 'b', 'c'`
- Используя встроенную функцию `tuple()`: `my_tuple = tuple([1, 2, 3, 'a', 'b', 'c'])`
- С помощью оператора упаковки:
`a = 1`
`b = 2`
`c = 3`
`my_tuple = a, b, c` # Эквивалентно `(a, b, c)`
- Из итерируемого элемента, например строки, с помощью функции `tuple()`:
`my_tuple = tuple('hello')`
Результат: `('h', 'e', 'l', 'l', 'o')`
- Можно создать пустой кортеж: `empty_tuple = ()`
- Если требуется кортеж, состоящий всего из одного элемента, то после элемента необходимо обязательно поставить запятую:
`single_element_tuple = (42,)`
Или так
`single_element_tuple = 42,`



Основные операции над кортежами

Доступ к элементам по индексам

Кортеж — это упорядоченная последовательность. Поэтому для доступа к его элементам можно использовать индексы.

Индексация начинается с нуля и заканчивается значением, равным длине кортежа, уменьшенным на единицу все так же как и в списках:

```
my_tuple = ('Cat', 'Turtle', 'Snake', 'Dog')
```

```
first_element = my_tuple[0]
```

```
second_element = my_tuple[1]
```

```
third_element = my_tuple[2]
```

```
print(first_element)
```

```
print(second_element)
```

```
print(third_element)
```

>>>

Cat

Turtle

Snake

Как видно из примера, действительно доступ по индексам работает, аналогично как и со списками

Для доступа к элементам с конца кортежа можно использовать отрицательные индексы. В этом случае индексация начинается с -1 и заканчивается значением, равным длине кортежа со знаком минус.



Основные операции над кортежами

Упаковка и распаковка

Упаковка (packing) и распаковка (unpacking) — это операции, которые позволяют создавать кортеж из набора значений и извлекать значения из кортежа в переменные. Рассмотрим их подробнее.

Упаковка кортежа. Значения 'Cat', 'Turtle', 'Snake', 'Dog' автоматически упаковываются в кортеж:

```
my_tuple = 'Cat', 'Turtle', 'Snake', 'Dog'
print(my_tuple) >>> ('Cat', 'Turtle', 'Snake', 'Dog')
```

Распаковка кортежа. Элементы кортежа извлекаются в переменные:

```
cat, turtle, snake, dog = my_tuple
print(cat, turtle, snake, dog) >>> Cat Turtle Snake Dog
```

Обмен значениями между переменными. С помощью упаковки и распаковки кортежей можно обменивать значения переменных между собой без использования временной переменной.

```
pet_1 = "Cat"
pet_2 = "Dog"

pet_1, pet_2 = pet_2, pet_1
print(f"Теперь pet_1 это: {pet_1}")
print(f"Теперь pet_2 это: {pet_2}") >>> Теперь pet_1 это: Dog
Теперь pet_2 это: Cat
```




Основные операции над кортежами

Сравнение

Кортежи в Python сравниваются покомпонентно начиная с первого элемента до первого неравенства или до конца переменной. Если кортежи имеют одинаковую длину и все соответствующие элементы равны, то они считаются равными:

пример 1, кортежи с одинаковыми значениями, одинаковым порядком и одинаковой длины:

```
my_tuple_1 = ('Cat', 'Turtle', 'Snake', 'Dog')
my_tuple_2 = 'Cat', 'Turtle', 'Snake', 'Dog'
print(my_tuple_1 == my_tuple_2)

>>> True
```

В результате получаем True так как кортежи идентичны

пример 2, кортежи с одинаковыми значениями но с разным порядком:

```
my_tuple_1 = ('Cat', 'Turtle', 'Snake', 'Dog')
my_tuple_2 = 'Dog', 'Turtle', 'Snake', 'Cat'
print(my_tuple_1 == my_tuple_2)

>>> False
```

В результате получаем False поскольку хоть элементы и одинаковые однако они стоят в разном порядке.

пример 3, кортежи с разной длины:

```
my_tuple_1 = ('Cat', 'Turtle', 'Snake', 'Dog')
my_tuple_2 = 'Cat', 'Turtle', 'Snake', 'Dog', 'Parrot'
print(my_tuple_1 == my_tuple_2)

>>> False
```

Опять получаем False, пусть часть элементов и одинакова, и порядок не нарушен, однако второй кортеж на 1 элемент длиннее



Основные операции над кортежами

Сравнение

Для сравнения используются стандартные операторы `==`, `!=`, `>`, `<`, `>=`, `<=`.

При сравнении используются правила:

- когда сравниваются числовые элементы, большим считается тот кортеж, у которого первый отличающийся элемент больше;
- когда сравниваются строки, сравнение происходит по порядку в алфавите;
- элементы различных типов сравнивать нельзя — получим ошибку `TypeError`;
- если длины кортежей различны, то кортеж с меньшей длиной считается наименьшим.

```
tuple_1 = (1, 2, 3)
tuple_2 = (1, 2, 4)
print(tuple_1 < tuple_2)
```

```
>>> True
```

True, так как третий элемент tuple_2 больше чем третий элемент tuple_1

```
tuple_3 = ('Cat', 'Dog')
tuple_4 = ('Cat', 'Turtle')
print(tuple_3 < tuple_4)
```

```
>>> True
```

True, так как второй элемент tuple_3 "меньше" tuple_4 по порядковому номеру первой отличающейся буквы

```
tuple_5 = ('Cat', 'Dog')
tuple_6 = ('Cat', 'Dog', 'Turtle')
print(tuple_5 < tuple_6)
```

```
>>> True
```

True, так длина tuple_6 больше чем длина tuple_5

```
tuple_7 = (1, 2, 3)
tuple_8 = ('Cat', 'Dog', 'Turtle')
print(tuple_7 < tuple_8)
```

```
>>> TypeError: '<' not supported between instances of 'int' and 'str'
```

Ошибка типа, т.к. нельзя сравнивать числа и строки



Основные операции над кортежами

Проверка принадлежности элемента

Чтобы проверить, принадлежит ли элемент кортежу, используется оператор `in`. Если элемент присутствует в кортеже, оператор возвращает `True`, в противном случае — `False`.

```
my_tuple = ('Cat', 'Turtle', 'Snake', 'Dog')
animal = input("Введите питомца: ")
if animal in my_tuple:
    print("Такой питомец есть в кортеже")
else:
    print("Такого питомца нет в кортеже")
```

>>>

```
Введите питомца: Cat
Такой питомец есть в кортеже
Введите питомца: Parrot
Такого питомца нет в кортеже
```

Я 2 раза запустил код и действительно, если элемент есть в кортеже то программа идет по пути `if` если элемента нет то программа идет по пути `else`

Перебор элементов

Для перебора элементов кортежа в Python используются циклы:

```
animals_tuple = ('Cat', 'Turtle', 'Snake')
for animal in animals_tuple:
    print(animal)
```

>>>

```
Cat
Turtle
Snake
```

Цикл работает так же как и цикл по спискам или множествам, берется элемент, и с ним выполняются те или иные действия



Основные операции над кортежами

Конкатенация и повторение

В Python для кортежей доступны операции конкатенации (сложения) и повторения (умножения). Эти операции могут использоваться при создании новых кортежей на основе существующих.

Конкатенация. Это объединение двух кортежей в один новый кортеж с помощью оператора +.

```
pets_tuple = ('Cat', 'Dog')
wild_animals_tuple = ('Lion', 'Wolf')
all_animals_tuple = pets_tuple + wild_animals_tuple
print(all_animals_tuple)

>>> ('Cat', 'Dog', 'Lion', 'Wolf')
```

Из двух кортежей мы получили один, но он содержит все элементы из каждого кортежа.

Повторение. Этот способ создания кортежа напоминает арифметическое умножение: справа от исходного кортежа нужно добавить оператор * и указать, сколько раз он повторится.

```
pets_tuple = ('Cat', 'Dog')
repeated_pets_tuple = pets_tuple * 3
print(repeated_pets_tuple)

>>> ('Cat', 'Dog', 'Cat', 'Dog', 'Cat', 'Dog')
```

Кортеж `pets_tuple`, повторился 3 раза и записался в новый кортеж `repeated_pets_tuple`

Функции для работы с кортежами





Функции для работы с кортежами

Определение длины, суммы, минимального и максимального элемента:

- длину кортежа можно получить функцией **len()**, которая возвращает количество элементов в кортеже

```
animals_tuple = 'Cat', 'Turtle', 'Snake', 'Dog'
nums_tuple = (1, 2, 3, 4, 5)
animals_tuple_length = len(animals_tuple)
nums_tuple_length = len(nums_tuple)
print(f'Длина кортежа зверей: {animals_tuple_length}.')
print(f'Длина кортежа чисел: {nums_tuple_length}.')
```

>>>

Длина кортежа зверей: 4.
Длина кортежа чисел: 5.

Данная функция работает аналогично, как и со строками, списками или множествами

- сумма элементов рассчитывается функцией **sum()** но она работает только с числами:

```
nums_tuple = (1, 2, 3, 4, 5)
sum_of_elements = sum(nums_tuple)
print(f'Сумма элементов кортежа: {sum_of_elements}.')
```

>>>

Сумма элементов кортежа: 15

Функция работает аналогично, как и со списками или множествами

- Минимальный и максимальный элементы кортежа можно получить с помощью функций **min()** и **max()**:

```
animals_tuple = 'Cat', 'Turtle', 'Snake', 'Dog'
nums_tuple = (1, 2, 3, 4, 5)
min_animal_element = min(animals_tuple)
max_animal_element = max(animals_tuple)
min_nums_element = min(nums_tuple)
max_nums_element = max(nums_tuple)
print(f'Минимальный элемент кортежа зверей: {min_animal_element}.')
print(f'Максимальный элемент кортежа зверей: {max_animal_element}.')
print(f'Минимальный элемент кортежа чисел: {min_nums_element}.')
print(f'Максимальный элемент кортежа чисел: {max_nums_element}.')
```

>>>

Минимальный элемент кортежа зверей: Cat.
Максимальный элемент кортежа зверей: Turtle.
Минимальный элемент кортежа чисел: 1.
Максимальный элемент кортежа чисел: 5.

Для строк максимальный символ с наибольшим порядковым номером и наоборот, для чисел самое большое или маленькое число



Функции для работы с кортежами

Сортировка кортежа. Элементы можно отсортировать по возрастанию или по убыванию. Так как кортежи относятся к неизменяемому типу данных, то кортеж с упорядоченными элементами нужно будет сохранить в новой переменной.

Для сортировки используется функция **sorted()**:

- **sorted(original_tuple)** создаёт новый список, содержащий отсортированные элементы из original_tuple;
- **tuple(...)** создаёт новый кортеж из отсортированного списка.

```
animals_tuple = 'Cat', 'Turtle', 'Snake', 'Dog'
nums_tuple = (3, 2, 1, 5, 4)
```

```
sorted_animals = sorted(animals_tuple)
sorted_nums = sorted(nums_tuple)
```

```
sorted_animals_tuple = tuple(sorted_animals)
sorted_nums_tuple = tuple(sorted_nums)
```

```
print(f'Исходный кортеж зверей: {animals_tuple}.')
print(f'Промежуточный результат сортировки: {sorted_animals}.')
print(f'Отсортированный кортеж зверей: {sorted_animals_tuple}.')
print(f'Исходный кортеж чисел: {nums_tuple}.')
print(f'Промежуточный результат сортировки: {sorted_nums}.')
print(f'Отсортированный кортеж чисел: {sorted_nums_tuple}.')
```

>>>

Исходный кортеж зверей: ('Cat', 'Turtle', 'Snake', 'Dog').

Промежуточный результат сортировки: ['Cat', 'Dog', 'Snake', 'Turtle'].

Отсортированный кортеж зверей: ('Cat', 'Dog', 'Snake', 'Turtle').

Исходный кортеж чисел: (3, 2, 1, 5, 4).

Промежуточный результат сортировки: [1, 2, 3, 4, 5].

Отсортированный кортеж чисел: (1, 2, 3, 4, 5).



Функции для работы с кортежами

- для сортировки по убыванию, необходимо задать дополнительный параметр **reverse=True**

```
animals_tuple = 'Cat', 'Turtle', 'Snake', 'Dog'
nums_tuple = (3, 2, 1, 5, 4)

reverse_sorted_animals = sorted(animals_tuple, reverse=True)
reverse_sorted_nums = sorted(nums_tuple, reverse=True)

reverse_sorted_animals_tuple = tuple(sorted_animals)
reverse_sorted_nums_tuple = tuple(sorted_nums)

print(f'Исходный кортеж зверей: {animals_tuple}.')
print(f'Промежуточная обратная сортировка: {reverse_sorted_animals}.')
print(f'Отсортированный кортеж зверей в обратном порядке: {reverse_sorted_animals_tuple}.')
print(f'Исходный кортеж чисел: {nums_tuple}.')
print(f'Промежуточная обратная сортировка: {reverse_sorted_nums}.')
print(f'Отсортированный кортеж чисел в обратном порядке: {reverse_sorted_nums_tuple}.')

      v   v   v

Исходный кортеж зверей: ('Cat', 'Turtle', 'Snake', 'Dog').
Промежуточная обратная сортировка: ['Turtle', 'Snake', 'Dog', 'Cat'].
Отсортированный кортеж зверей в обратном порядке: ('Cat', 'Dog', 'Snake', 'Turtle').
Исходный кортеж чисел: (3, 2, 1, 5, 4).
Промежуточная обратная сортировка: [5, 4, 3, 2, 1].
Отсортированный кортеж чисел в обратном порядке: (1, 2, 3, 4, 5).
```

Как видно из примеров, с прямой и обратной сортировкой, оно работает так же как и со списками, но поскольку кортеж это неизменяемый тип данных, то функция `sorted()` преобразовывает кортеж в изменяемый тип - список, после чего сортирует элементы и для того чтобы получить в итоге отсортированный кортеж мы должны преобразовать отсортированный список в кортеж при помощи функции `tuple()`, результат записать в новую переменную



Функции для работы с кортежами

Срезы в кортежах

Срезы используются для извлечения из кортежей подмножества элементов. Для этого необходимо указать начальный и конечный индекс, а также шаг среза (если это необходимо).

Для определения начала и конца среза можно использовать положительные и отрицательные индексы:

- левый индекс указывает на элемент, с которого начинается срез;
- правый индекс указывает на элемент, которым заканчивается срез, этот элемент в срез не включается.

Получим срез, указав положительные индексы:

```
animals_tuple = ('Cat', 'Turtle', 'Snake', 'Dog', 'Owl', 'Parrot')
slice_positive = animals_tuple[1:5]
print(f'Срез с положительными индексами: {slice_positive}')
```

>>> Срез с положительными индексами: ('Turtle', 'Snake', 'Dog', 'Owl')

Получим срез, указав отрицательные индексы:

```
animals_tuple = ('Cat', 'Turtle', 'Snake', 'Dog', 'Owl', 'Parrot')
slice_negative = animals_tuple[-5:-1]
print(f'Срез с отрицательными индексами: {slice_negative}')
```

>>> Срез с отрицательными индексами: ('Turtle', 'Snake', 'Dog', 'Owl')



Функции для работы с кортежами

Получим срез от начала кортежа, указав только конец среза:

```
animals_tuple = ('Cat', 'Turtle', 'Snake', 'Dog', 'Owl', 'Parrot')
slice_from_start_positive = animals_tuple[:4]
slice_from_start_negative = animals_tuple[:-2]
print(f'Срез от начала до 4-го элемента: {slice_from_start_positive}')
print(f'Срез от начала до -2-го элемента: {slice_from_start_negative}')
```

>>> Срез от начала до 4-го элемента: ('Cat', 'Turtle', 'Snake', 'Dog')
Срез от начала до -2-го элемента: ('Cat', 'Turtle', 'Snake', 'Dog')

Получим срез до конца кортежа, указав только начало среза:

```
animals_tuple = ('Cat', 'Turtle', 'Snake', 'Dog', 'Owl', 'Parrot')
slice_to_end_positive = animals_tuple[3:]
slice_to_end_negative = animals_tuple[-3:]
print('Срез от 3-го элемента до конца:', slice_to_end_positive)
print('Срез от -3-го элемента до конца:', slice_to_end_negative)
```

>>> Срез от 3-го элемента до конца: ('Dog', 'Owl', 'Parrot')
Срез от -3-го элемента до конца: ('Dog', 'Owl', 'Parrot')

Получим срез с желаемым шагом (в примере 2 но он может быть любой):

```
animals_tuple = ('Cat', 'Turtle', 'Snake', 'Dog', 'Owl', 'Parrot')
slice_with_step_1 = animals_tuple[::2]
slice_with_step_2 = animals_tuple[1:4:2]
print(f'Срез с шагом по всему кортежу: {slice_with_step_1}')
print(f'Срез с шагом от 1го до 4го индекса: {slice_with_step_2}')
```

>>> Срез с шагом по всему кортежу: ('Cat', 'Snake', 'Owl')
Срез с шагом от 1го до 4го индекса: ('Cat', 'Snake', 'Owl')



Функции для работы с кортежами

Преобразование в другие типы данных

В Python можно преобразовывать кортежи в другие типы данных: списки, строки, множества или словари.

Преобразование в список.

```
animals_tuple = ('Cat', 'Turtle', 'Snake', 'Dog')
```

```
animals_list = list(animals_tuple)
```

```
print(f'Кортеж: {animals_tuple}')
```

```
print(f'Список: {animals_list}')
```

Преобразование в строку.

```
animals_tuple = ('Cat', 'Turtle', 'Snake', 'Dog')
```

```
animals_string = ', '.join(animals_tuple)
```

```
print(f'Кортеж: {animals_tuple}')
```

```
print(f'Строка: {animals_string}')
```

Преобразование в множество (повторяющиеся элементы будут удалены).

```
animals_tuple = ('Cat', 'Dog', 'Turtle', 'Cat', 'Dog')
```

```
animals_set = set(animals_tuple)
```

```
print(f'Кортеж: {animals_tuple}')
```

```
print(f'Множество: {animals_set}')
```

Преобразование в словарь парами ключ: значение.

```
animals_tuple = (('Cat', 'Кот'), ('Dog', 'Пес'), ('Turtle', 'Черепаха'))
```

```
animals_dict = dict(animals_tuple)
```

```
print(f'Кортеж: {animals_tuple}')
```

```
print(f'Словарь: {animals_dict}')
```

>>>

Кортеж: ('Cat', 'Turtle', 'Snake', 'Dog')

Список: ['Cat', 'Turtle', 'Snake', 'Dog']

>>>

Кортеж: ('Cat', 'Turtle', 'Snake', 'Dog')

Строка: Cat, Turtle, Snake, Dog

>>>

Кортеж: ('Cat', 'Dog', 'Turtle', 'Cat', 'Dog')

Множество: {'Cat', 'Dog', 'Turtle'}

>>>

Кортеж: (('Cat', 'Кот'), ('Dog', 'Пес'), ('Turtle', 'Черепаха'))

Словарь: {'Cat': 'Кот', 'Dog': 'Пес', 'Turtle': 'Черепаха'}



Функции для работы с кортежами

Как можно изменить элементы кортежа?

Хотя кортежи относятся к неизменяемым элементам, есть несколько трюков, которые позволят их изменять.

- переводим кортеж в список, делаем необходимые преобразования, а затем превращаем полученный список в кортеж (однако такие функции лучше скрывать от обычных пользователей):

```
animals_tuple = ('Cat', 'Turtle', 'Snake', 'Dog')
animals_list = list(animals_tuple)
animals_list[2] = 'Python'
animals_list.append('Owl')
animals_list.remove('Turtle')
updated_tuple = tuple(animals_list)
print(f'Исходный кортеж: {animals_tuple}')
print(f'Обновлённый кортеж: {updated_tuple}')
```

>>> Исходный кортеж: ('Cat', 'Turtle', 'Snake', 'Dog')
Обновлённый кортеж: ('Cat', 'Python', 'Dog', 'Owl')

В данном случае мы можем делать все то что мы можем делать со списками (изменять, добавлять, удалять)

- используя срезы собрать из них новый кортеж

```
animals_tuple = ('Cat', 'Turtle', 'Snake', 'Dog')
updated_tuple = animals_tuple[:2] + ('Python',) + animals_tuple[3:] + ('Owl',)
print('Исходный кортеж:', animals_tuple)
print('Обновлённый кортеж:', updated_tuple)
```

>>> Исходный кортеж: ('Cat', 'Turtle', 'Snake', 'Dog')
Обновлённый кортеж: ('Cat', 'Turtle', 'Python', 'Dog', 'Owl')

А в этом примере, мы выбирая нужные индексы заменяем значение внутри, и добавляем новое имейте в виду мы кортежи суммируем с кортежами т.к. ('Python',) и ('Owl',) это тоже кортежи из одного элемента (для этого мы и берем значение в скобки и ставим запятую)

Множества





Что такое множество?

Множество - это специальный тип данных который используется для проверки уникальности и принадлежности элементов. Характеристиками данного типа данных являются:

Тип **set** является **ИЗМЕНЯЕМЫМ** множеством, содержимое может быть изменено специальных методов (они будут рассмотрены далее). Поскольку тип set является изменяемым, он не имеет хеш-значения и не может использоваться ни как ключ словаря, ни как элемент другого множества его характеристики:

- неупорядоченность - элементы внутри множеств находятся на произвольных местах;
- изменяемость - можно добавлять или удалять элементы;
- уникальность - содержаться только не повторяющиеся значения.

Тип **frozenset** является **НЕизменяемым и хешируемым** множеством, его содержимое не может быть изменено после его создания, поэтому он может использоваться как ключ словаря или как элемент другого множества, его характеристики:

- неупорядоченность - элементы внутри множеств находятся на произвольных местах;
- неизменяемость - нельзя добавлять или удалять элементы;
- уникальность - содержаться только не повторяющиеся значения.

Как **set** так и **frozenset**

- Множества не поддерживают сортировку.
- Множества не поддерживают индексирование элементов



Что такое множество?

Существуют специальные операции над множествами, например у нас есть студенты которые изучают Python и студенты которые изучают английский, и мы можем объединить этих студентов в одно множество.

Для чего и почему они используются?

Множества работают быстрее списков и поэтому они полезны в работе с большими наборами данных, которые нужно делить, объединять или вычитать между собой. Могут использоваться в следующих задачах:

- работа с анализом данных;
- сегментирование целевой аудитории в рекламе;
- сложные фильтры в каталогах и т.п.

Однако у множеств есть и **существенный недостаток** множества занимают гораздо больший объем памяти, чем списки и кортежи.



Множество, для чего можно использовать

Set - это множество, его еще называют набор:

- **первое** на чем нужно заострить внимание - в нем не может быть повторяющихся элементов;
- **второе** оно пишется в фигурных {} скобках.

```
my_animals = {"Cat", "Dog", "Turtle", "Cat", "Dog"}
print(f"Длина множества: {len(my_animals)}")
print(f"Элементы множества: {my_animals}")
```

>>> Длина множества: 3
Элементы множества: {'Dog', 'Cat', 'Turtle'}

Как видим из примера выше, хоть изначально мы передали 5 элементов, и некоторые из них повторялись, в результате работы программы этих элементов осталось 3 и остались только уникальные элементы.

Эта особенность множеств позволяет нам уникализировать списки (то есть убирать из них повторяющиеся значения).

```
1 my_animals = ["Cat", "Dog", "Turtle", "Cat", "Dog"]
2 my_unique_animals = set(my_animals)
3 print(my_unique_animals)
4 print(list(my_unique_animals))
```

>>> {'Dog', 'Cat', 'Turtle'}
['Dog', 'Cat', 'Turtle']

В пример мы передали список(1), преобразовав его во множество при помощи функции **set()** (2) и вывели в консоль полученные результаты(3), как получившееся множество так и множество преобразованное обратно в список но уже с неповторяющимися элементами(4).



Множество, для чего можно использовать

Множества не упорядочены - поэтому у них нет индексов и слайсов

```
my_animals = {'Cat', 'Turtle', 'Dog'}  
print(my_animals[1])
```

>>> **TypeError: 'set' object is not subscriptable**

Запустив этот код мы получаем ошибку типа, которая нам в данном случае сообщает что у объектов внутри множества нет подписи (в данном случае индекса), т.к. внутри множества объекты не привязаны к какому-то своему месту поэтому мы не сможем получить нужный нам объект таким образом.

А теперь не очень хорошая новость если передать в функцию множество (**set()**) строку то оно разобьет ее по буквам (а не создаст множество с 1м элементом).

```
my_animal = set('Turtle')  
print(my_animal)
```

>>> {'r', 'u', 'e', 'l', 'T', 't'}

А если мы просто передадим несколько элементов через запятую, то мы получим следующий результат:

```
my_animals = set('Turtle', 'Cat', 'Dog')  
print(my_animals)
```

>>> **TypeError: set expected at most 1 argument, got 3**

И это ошибка типа, которая нам сообщает что функция множество ожидает как максимум 1 аргумент а получила она 3

Методы множеств



Какие есть стандартные методы есть у множеств?

Множества имеют ряд методов с которые позволяют производить с ними те или иные операции (кроме цикла недоступны для типа frozenset):

- метод **.add()** - позволяет добавить элемент в наше множество (не спрашивайте почему не `.append()`)

```
my_animals = {'Cat', 'Dog'}
my_animals.add('Turtle')      >>> {'Dog', 'Turtle', 'Cat'}
print(my_animals)
```

И да при помощи метода `.add()` мы добавили еще 1 элемент в наше множество, причем этот элемент может оказаться в любом месте.

- метод **.pop()** - удаляет элемент из множества, при этом надо знать то, что будет удален какой-то случайный элемент:

```
my_animals = {'Dog', 'Turtle', 'Cat'}
my_animals.pop()              >>> {'Turtle', 'Dog'}
print(my_animals)             {'Turtle', 'Cat'}
```

Для наглядности я запустил этот код 2 раза и как видите после операции удаления мы получили разные результаты.

- метод **.discard()** - удаляет элемент из множества, в скобки требуется передать тот элемент который мы бы хотели удалить, если такого элемента нет то код просто отработает не выдавая ошибку:

```
my_animals = {'Dog', 'Turtle', 'Cat'}
my_animals.discard('Turtle')  >>> {'Dog', 'Cat'}
print(my_animals)
```

Действительно, мы удалили из нашего множества именно тот элемент который мы передали методу `.discard()`



Какие есть стандартные методы есть у множеств?

- метод `.remove()` - также удаляет элемент из множества, в скобки требуется передать тот элемент который мы бы хотели удалить, однако если элемента в множестве нет то мы получим ошибку

```
my_animals = {'Dog', 'Turtle', 'Cat'}  
my_animals.remove('Turtle')  
print(my_animals) >>> {'Dog', 'Cat'}
```

Мы передали в метод `.remove()` **существующий** элемент 'Turtle' и данный элемент был удален из нашего множества.

```
my_animals = {'Dog', 'Turtle', 'Cat'}  
my_animals.remove('Parrot')  
print(my_animals) >>> KeyError: 'Parrot'
```

А тут мы передали в метод `.remove()` **не существующий** элемент 'Parrot' и получили ошибку ключа (в данном случае ключ это и есть переданный элемент)

Эту ошибку можно обойти при помощи условных конструкций, если мы хотим проверить наличие элемента в множестве и при его наличии удалить, а если его нет совершить иное действие.

```
my_animals = {'Dog', 'Turtle', 'Cat'}  
animal_to_remove = 'Parrot'  
if animal_to_remove in my_animals:  
    my_animals.remove(animal_to_remove)  
else:  
    print('Данный элемент отсутствует в множестве') >>> Данный элемент отсутствует в множестве
```

В этом примере мы обходим данную ошибку при помощи ключевого слова **in**, и так как элемента нет код выполняется по условию **else**:



Какие есть стандартные методы есть у множеств?

- метод **set.clear()** удаляет все элементы из множества set. Метод очищает множество и не возвращает никакого результата.

```
my_animals_set = {"Cat", "Dog", "Turtle", "Owl", "Snake"}
my_animals_set.clear()
print(my_animals_set)
```

>>> set()

Как видим все элементы из нашего множества были удалены, и теперь это просто пустое множество

- по множеству можно запустить и цикл

```
my_animals = {'Dog', 'Turtle', 'Cat'}
for my_animal in my_animals:
    print(my_animal)
```

>>>

Turtle
Dog
Cat

Цикл работает как и обычно, с каждым элементом внутри нашего множества выполняется то или иное запрограммированное действие, но в отличие от цикла по списку элементы берутся в случайном порядке

Особые методы множеств (доступны и для `set` и для `frozenset`)

Конечно у множеств есть и свои уникальные методы:

- **`set.union()`** позволяет объединить множество с двумя или более последовательностями поддерживающих итерирование. Метод возвращает новое множество с элементами из множества `set` и элементами вставленными из всех итерируемых объектов `*other`. При выполнении операции объединения, дубликаты игнорируются.
- **`set.intersection()`** позволяет найти пересечение множества с одной или более последовательностями поддерживающих итерирование. Метод возвращает новое множество с элементами, общими для множества `set` и всех итерируемых объектов `*other`. При выполнении операции пересечения, дубликаты игнорируются.
- **`set.difference()`** позволяет получить элементы множества, которых нет в одной или более последовательности поддерживающих итерирование. Метод возвращает новое множество с уникальными элементами множества `set`, которых нет во всех итерируемых объектах `*other`. При выполнении операции вычитания, дубликаты игнорируются.
- **`set.issubset()`** - позволяет проверить находится ли каждый элемент множества `set` в последовательности `other`. Метод возвращает `True`, если множество `set` **является подмножеством** итерируемого объекта `other`, если нет, то вернет `False`.
- **`set.issuperset()`** позволяет проверить находится ли каждый элемент последовательности `other` в множестве `set`. Метод возвращает `True`, если множество `set` является надмножеством итерируемого объекта `other`, если нет, то вернет `False`.

Особые методы множеств (доступны и для `set` и для `frozenset`)

- Метод **`set.isdisjoint()`** возвращает `True`, если множество `set` не имеет общих элементов с итерируемым объектом `other`. Итерируемый объект `other`, это объект поддерживающий итерацию по своим элементам, может быть список, кортеж, другое множество
- Метод **`set.symmetric_difference()`** позволяет исключить из результата общие элементы для множества и последовательности, операцию еще называют симметричной разностью. Метод возвращает новое множество, в котором нет одинаковых элементов, встречающихся одновременно в множестве `set` и итерируемом объекте `other`. При выполнении операции симметричного разности, дубликаты игнорируются.
- **Проверка множества на правильное подмножество в Python** - математический оператор `<` (меньше) позволяет проверить, является ли множество `set` подходящим подмножеством другого множества `other`. Множество меньше другого тогда и только тогда, когда первое множество является правильным подмножеством второго
- **Проверка множества на правильное надмножество в Python** - математический оператор `>` (больше) позволяет проверить, является ли множество `set` правильным надмножеством другого множества `other`, то есть выполняются ли условия `set >= other` и `set != other`.
- **`set.copy()`** - возвращает копию множества `set`.

Подробнее об этих методах вы можете узнать по следующей ссылке:

- <https://docs-python.ru/tutorial/obschie-operatsii-mnozhestvami-set-frozenset-python/>



Для чего целесообразно использовать множества?

Множества используются для специфических задач пересечения. Вот какие методы существуют:

- **.union()** - соединяют множества, например у нас есть какие-то животные и какие-то животные продаются в зоомагазине и мы хотим получить всех животных но без повторений, для этого мы к нашему первому множеству применяем метод `.union()` в скобки к которому передаем второе множество (или другую итерируемую последовательность):

```
my_animals = {'Cat', 'Dog'}
shop_animals = {'Cat', 'Turtle', 'Snake'}
all_animals = my_animals.union(shop_animals)
print(all_animals)

>>> {'Turtle', 'Snake', 'Dog', 'Cat'}
```

Как видим из примера мы получили третье множество где объединили 2 наших начальных множества.

- **.intersection()** - это пересечение, например какие животные есть и у меня и в зоомагазине, аналогично применяем данный метод к первому множеству, а в скобки к методу передаем второе множество (или другую итерируемую последовательность):

```
my_animals = {'Cat', 'Dog'}
shop_animals = {'Cat', 'Turtle', 'Snake'}
same_animals = my_animals.intersection(shop_animals)
print(same_animals)

>>> {'Cat'}
```

И действительно элемент 'Cat' есть в обоих множествах, следовательно мы его и получаем в итоге работы нашего кода



Для чего целесообразно использовать множества?

- **.difference()** - это вычитание, тут мы узнаем каких животных из первого множества нет во втором множестве, в нашем примере что есть у нас и чего нет в зоомагазине:

```
my_animals = {'Cat', 'Dog'}
shop_animals = {'Cat', 'Turtle', 'Snake'}
only_my_animals = my_animals.difference(shop_animals)
print(only_my_animals)

>>> {'Dog'}
```

Мы выявили разницу между my_animals и shop_animals, причем первым передали именно my animals тем самым как бы спросив а что есть уникальное именно в my_animals

- и наоборот то что есть в магазине и то чего нет у нас:

```
my_animals = {'Cat', 'Dog'}
shop_animals = {'Cat', 'Turtle', 'Snake'}
only_shop_animals = shop_animals.difference(my_animals)
print(only_shop_animals)

>>> {'Snake', 'Turtle'}
```

А тут наоборот, первым передали shop_animals спрашивая что есть уникальное именно в shop_animals

- **.issubset()** - данный метод возвращает булев тип (True или False). True если то что есть у меня в полном объеме есть в зоомагазине или False если у меня есть что-то такое чего в зоомагазине нет.

```
my_animals = {'Cat', 'Dog'}
shop_animals = {'Cat', 'Turtle', 'Snake', 'Dog'}
result = my_animals.issubset(shop_animals)
print(result)

>>> True

my_animals = {'Cat', 'Dog'}
shop_animals = {'Cat', 'Turtle', 'Snake'}
result = my_animals.issubset(shop_animals)
print(result)

>>> False
```

Как видно из примеров в первом случае в зоомагазине есть и те элементы которые есть у меня поэтому мы и получили True, а во втором случае в зоомагазине нет элемента 'Dog' поэтому наш результат False.



Для чего целесообразно использовать множества?

- **.issuperset()** - позволяет проверить находится ли каждый элемент последовательности other в множестве set. Метод возвращает True, если множество set является надмножеством итерируемого объекта other, если нет, то вернет False.

```
animals_set = {"Cat", "Turtle", "Snake", "Dog"}
animals_list = ["Cat", "Dog"]
animals_tuple = ("Cat", "Dog")

>>> True
>>> True

print(animals_set.issuperset(animals_list))
print(animals_set.issuperset(animals_tuple))
```

В нашем примере **все элементы списка и словаря присутствуют в множестве animals_set**, следовательно **результат True**, если будут хоть какие-то различия результат будет False

- **.isdisjoint()** - возвращает True, если множество set не имеет общих элементов с итерируемым объектом other. Итерируемый объект other, это объект поддерживающий итерацию по своим элементам, может быть список, кортеж, другое множество (вообще ни одного общего элемента)

```
animals_set = {"Cat", "Turtle", "Snake", "Dog"}
animals_list = ["Wolf", "Lion"]
animals_tuple = ("Wolf", "Lion")

>>> True
>>> True

print(animals_set.isdisjoint(animals_list))
print(animals_set.isdisjoint(animals_tuple))
```

Этот метод работает точно противоположным образом, если **НИ ОДНОГО элемента** в списке или кортеже **нет в множестве мы получим True**, если хотя бы один то получим False.



Операции с изменяемым множеством `set`

Перечисленные ранее методы работают как для `set` так и для `frozenset`, однако есть методы которые доступны только для изменяемого множества

- Метод **`set.update()`** позволяет добавить в множество `set`, элементы из одной или более последовательности поддерживающих итерирование. Метод возвращает обновленное множество `set` с добавленными элементами из всех итерируемых объектов `*other`
- Метод **`set.intersection_update()`** позволяет сохранить в множестве `set` только те элементы, которые присутствуют одновременно во всех объектах, участвующих в операции.
- Метод **`set.difference_update()`** позволяет удалить элементы из множества `set`, которые присутствуют во всех сравниваемых объектах.
- Метод **`set.symmetric_difference_update()`** позволяет изменить множество `set` так, что оно будет содержать уникальные элементы, встречающиеся в самом множестве и последовательности `other`.



Операции с изменяемым множеством set примеры:

```
shop_animals = {"Cat", "Turtle", "Snake", "Parrot"}
wild_animals = {"Fox", "Owl", "Snake", "Turtle"}

shop_animals.update(wild_animals)
print(shop_animals)

shop_animals = {"Cat", "Turtle", "Snake", "Parrot"}
wild_animals = {"Fox", "Owl", "Snake", "Turtle"}

shop_animals.intersection_update(wild_animals)
print(shop_animals)

shop_animals = {"Cat", "Turtle", "Snake", "Parrot"}
wild_animals = {"Fox", "Owl", "Snake", "Turtle"}

shop_animals.difference_update(wild_animals)
print(shop_animals)

shop_animals = {"Cat", "Turtle", "Snake", "Parrot"}
wild_animals = {"Fox", "Owl", "Snake", "Turtle"}

shop_animals.symmetric_difference_update(wild_animals)
print(shop_animals)
```

>>> {'Snake', 'Owl', 'Fox', 'Turtle', 'Parrot', 'Cat'}

>>> {'Snake', 'Turtle'}

>>> {'Parrot', 'Cat'}

>>> {'Owl', 'Fox', 'Parrot', 'Cat'}

Во всех наших примерах мы обновляем наше базовое множество и это необходимо учитывать, поэтому применять их нужно с оглядкой, особенно если вам важны базовые данные.



Множество применение:

У нас есть какие-то домашние питомцы, какие-то питомцы продаются в магазине а какие-то есть в питомнике, необходимо узнать:

- какие питомцев которые есть в магазине но нет у меня;
- какие питомцы есть как у меня так и в питомнике;
- каких моих питомцев нет ни в магазине ни в питомнике;
- какие у меня будут питомцы если я посету и магазин и питомник.

```
my_animals = {'Cat', 'Parrot', 'Owl'}
shop_animals = {'Turtle', 'Parrot', 'Rat'}
shelter_animals = {'Dog', 'Cat'}

print(shop_animals.difference(my_animals))
print(my_animals.intersection(shelter_animals))
not_my_animals = shop_animals.union(shelter_animals)
print(my_animals.difference(not_my_animals))
# print(my_animals.difference(shop_animals.union(shelter_animals)))
print(my_animals.union(not_my_animals))
```

>>>

```
{'Rat', 'Turtle'}
{'Cat'}
{'Owl'}
{'Parrot', 'Dog', 'Rat', 'Owl', 'Turtle', 'Cat'}
```